

EXPERIMENT 3

1.FUNDAMENTALS OF ERROR AND DEBBUGING

KEY DEFINITIONS:

1. Error: An **error** is a mistake made by the programmer while writing the source code.
2. Bug : A **bug** is a problem in a program that makes it give the wrong output or behave in an unexpected way.
3. Debugging: **Debugging** is the process of finding, understanding, and fixing bugs in a program.

Error Type	Description	Primary Cause	Example
Syntax Error	The code does not follow the rules of the C language.	Missing semicolon (;), missing braces {}, spelling mistakes in keywords.	int a = 10 (missing ;)
Compilation / Type Error	The code uses wrong data types or undeclared variables.	Using incompatible data types or variables that are not declared.	int *p = "hello";
Linker Error	The compiler creates object files, but the linker cannot	Function is declared but not defined, or required library is missing.	undefined reference to 'main'

Error Type	Description	Primary Cause	Example
	combine them.		
Runtime Error	The program crashes while running.	Division by zero, invalid memory access.	int x = 10 / 0;
Logical Error	The program runs without crashing, but gives the wrong output.	Wrong formula or incorrect loop condition.	for(i=0; i<5; i++) instead of for(i=0; i<=5; i++)

Why Does a Program Compile Successfully but Produce Incorrect Output?

The compiler only checks whether your code follows the **syntax** and **type rules** of the C language. It **cannot understand what you actually want the program to do**.

Example: `sum = a – b` instead of `sum=a-b`

This code is **correct in C syntax**, so it compiles successfully. But the output will be wrong because the logic is incorrect.

CASE 1 SYNTAX AND COMPILATION ERRORRS

EXAMPLE CODE

```
#include <stdio.h>
```

```
int main()
```

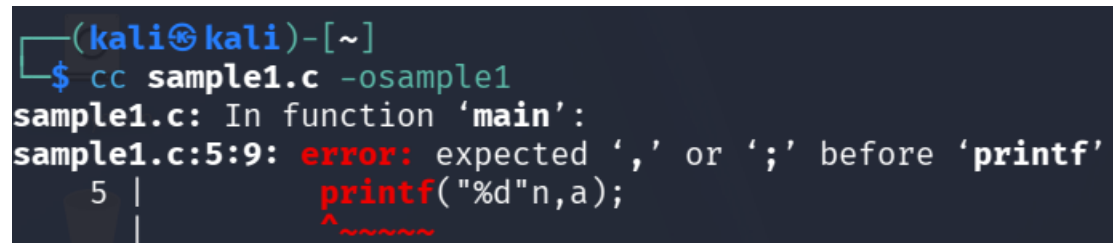
```
{
```

```
    int a = 10
```

```
    printf("%d\n", a);
```

```
    return 0;
}
```

COMPILATION COMAND AND OUTPUT



```
(kali㉿kali)-[~]
$ cc sample1.c -o sample1
sample1.c: In function 'main':
sample1.c:5:9: error: expected ',' or ';' before 'printf'
   5 |     printf("%d"n,a);
     |               ^~~~~
```

BREAKDOWN OF ERROR MESSAGE

- **Filename:** sample1.c
- **Line Number:** Line 5
- **Column Number:** Column 5
- **Type:** Error
- **Reason:** The compiler expected a comma , or a semicolon ; before the printf statement.

CASE 2 COMPILATION WARNINGS (-WALL,-WEXTRA)

A warning is a message from the compiler that tells you something in your code may cause problems. The program can still compile and run, but ignoring warnings can lead to bugs later.

EXAMPLE CODE

```
#include <stdio.h>
```

```
int main()
{
    int a;
    printf("Hello\n");
    return 0;
}
```

```
(kali㉿kali)-[~]  
$ cc sample2.c -osample2  
  
(kali㉿kali)-[~]  
$ ./sample2  
Hello  
  
(kali㉿kali)-[~]  
$ cc -Wall -wextrsample2.c  
cc: error: unrecognized command-line option '-wextrsample2.c'  
cc: fatal error: no input files  
compilation terminated.
```

- -Wall: Enables most common compiler warnings
- -Wextra: Enables additional warnings that are not included in -Wall.

CASE 3 PRINTF-BASED DEBUGGING ((LOGICAL BUG)

Before using advanced debugging tools like GDB, you can use `printf()` statements to check how your program is running. This helps you find logical errors by displaying the values of variables during execution.

EXAMPLE CODE

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int i, sum = 0;
```

```
    for(i = 0; i < 5; i++)
```

```
    {
```

```
        printf("DEBUG: entering iteration i = %d\n", i);
```

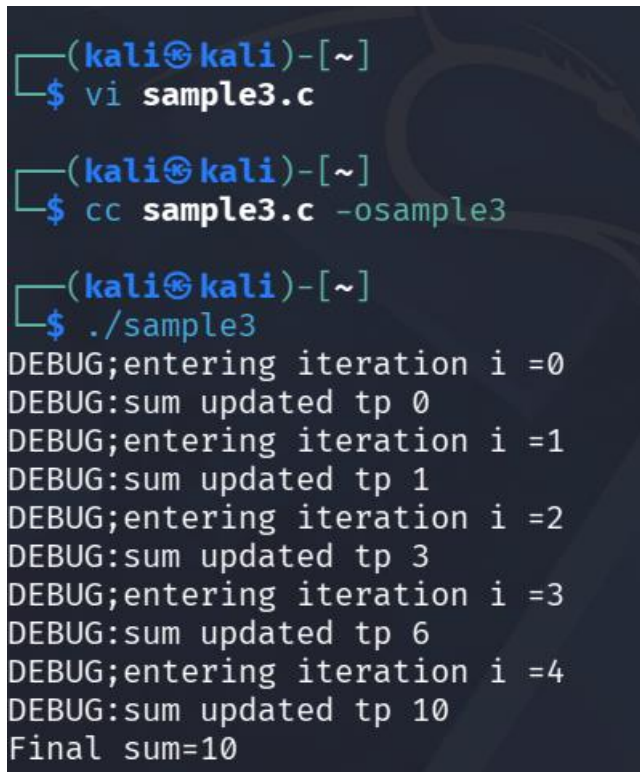
```
        sum = sum + i;
```

```
        printf("DEBUG: sum updated to = %d\n", sum);
```

```
    }
```

```
printf("Final Sum = %d\n", sum);

return 0;
}
```



```
(kali㉿kali)-[~]
$ vi sample3.c

(kali㉿kali)-[~]
$ cc sample3.c -osample3

(kali㉿kali)-[~]
$ ./sample3
DEBUG;entering iteration i =0
DEBUG:sum updated tp 0
DEBUG;entering iteration i =1
DEBUG:sum updated tp 1
DEBUG;entering iteration i =2
DEBUG:sum updated tp 3
DEBUG;entering iteration i =3
DEBUG:sum updated tp 6
DEBUG;entering iteration i =4
DEBUG:sum updated tp 10
Final sum=10
```

- **Introduction to GDB (GNU Debugger)**

- Background: What is GNU and GDB?**

- GNU**

- GNU** stands for "**GNU's Not Unix!**". It is a free and open-source operating system project started by **Richard Stallman** in **1983**. Its goal is to provide a completely free Unix-like operating system.

- GDB (GNU Debugger)**

- GDB (GNU Debugger)** is a debugging tool used to find and fix errors in C programs. It allows programmers to:

- Check the values of variables.
 - Set breakpoints to stop program execution.
 - Run the program step by step.
 - Find the cause of crashes and logical errors.

Common GDB Commands

Command	Short Form	Purpose
break <line/function>	b	Sets a breakpoint at a specific line or function.
run	r	Starts the program from the beginning.
next	n	Executes the next line without entering a function.
step	s	Executes the next line and enters the function if there is one.
print <variable>	p	Displays the current value of a variable.
display <variable>	disp	Automatically shows the variable value after every step.
continue	c	Continues program execution until the next breakpoint or the end of the program.
list	l	Displays the source code around the current line.
info locals	info loc	Shows all local variables and their current values.
backtrace	bt	Displays the sequence of function calls that led to the current point.
info breakpoints	i b	Shows all active breakpoints.
delete <id>	d	Deletes a breakpoint.

Command	Short Form	Purpose
disable <id>	dis	Temporarily disables a breakpoint.
enable <id>	en	Enables a disabled breakpoint again.
quit	q	Exits GDB.

CASE 4 DEBBUGING A LOGICAL ERROR USING GDB

A logical error means the program runs without crashing, but it gives the wrong output. GDB helps us find where the logic is incorrect by checking variable values step by step.

```
#include <stdio.h>    // Line 1

int main()           // Line 2
{                    // Line 3
    int i, sum = 0;   // Line 4

    for(i = 1; i < 5; i++) // Line 5 (Logical Bug: should be i <= 5)
    {                // Line 6
        sum = sum + i; // Line 7
    }                // Line 8

    printf("Sum = %d\n", sum); // Line 9
    return 0;                 // Line 10
}
```

```

(kali㉿kali)-[~]
$ vi sample4.c

(kali㉿kali)-[~]
$ cc sample4.c -osample4

(kali㉿kali)-[~]
$ ./sample4
Sum=10

(kali㉿kali)-[~]
$ cc-g sample4.c -osample4
cc-g: command not found

(kali㉿kali)-[~]
$ cc -g sample4.c -osample4

(kali㉿kali)-[~]
$ gdb ./sample4
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.

```

```

Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample4 ...
(gdb) list
1      #include<stdio.h>
2      int main()
3      {
4          int i,sum=0;
5          for(i=1;i<5;i++)
6          {
7              sum=sum+i;
8          }
9          printf("Sum=%d\n",sum);
10         return 0;

```



```

(gdb) break 7
Breakpoint 1 at 0x1151: file sample4.c, line 7.
(gdb) run
Starting program: /home/kali/sample4
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample4.c:7
7          sum=sum+i;
(gdb) print i
$1 = 1
(gdb) display sum
1: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum=sum+i;
1: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum=sum+i;
1: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum=sum+i;
1: sum = 6

```

The loop ends when **i becomes 5**, so the loop body is **not executed for i = 5**.

CASE 5 DEBBUGING FUNCTIONS (NEXT VS STEP)

When debugging a program that contains functions, GDB provides two useful commands:

- **next** – Executes the current line. If the line contains a function call, the function runs completely without entering it.
- **step** – Executes the current line and enters the function so you can debug it line by line.

EXAMPLE CODE

```
#include <stdio.h>
```

```
int square(int x)
```

```
{
```

```
    int result1;
```

```
    result1 = x * x;
```

```
    return result1;
```

```
}
```

```
int main()
```

```
{
```

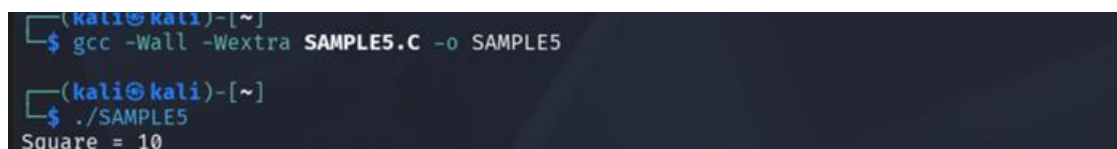
```
    int result, n = 5;
```

```
    result = square(n);
```

```
    printf("Square = %d\n", result);
```

```
    return 0;
```

```
}
```

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The first command is \$ gcc -Wall -Wextra SAMPLE5.C -o SAMPLE5. The second command is \$./SAMPLE5. The output is Square = 10.

```
(kali@kali)-[~]  
$ gcc -Wall -Wextra SAMPLE5.C -o SAMPLE5  
  
(kali@kali)-[~]  
$ ./SAMPLE5  
Square = 10
```

:stepping into with the step

(gdb)break main

(gdb)run

(gdb)next

(gdb)step

Square (x=5) at sample5.c:5

```
5 result= x * x;
```

```
(gdb)print x
```

```
$1=5
```

```
(gdb)next
```

```
(gdb)print result 1
```

```
$2=25
```

Error type:

Debugger demonstration

Error message:

No code error,demonstrates stepping

Explanation:

Use next to step over and step to step into functions

CASE 6 DEBBUGGING RUNTIME ERRORS USING BACKTRACE (backtrace/bt)

EXAMPLE CODE(SAMPLE6.C)

```
#include <stdio.h>
```

```
int divide(int a, int b)
```

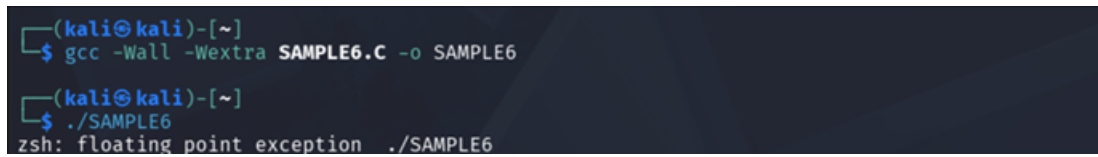
```
{ return a / b; }
```

```
int calculate(int x)
```

```
{ return divide(x, 0); } int main()
```

```
{ int result = calculate(10);
```

```
printf("Result: %d\n", result);  
  
return 0; }
```



```
(kali㉿kali)-[~]  
$ gcc -Wall -Wextra SAMPLE6.C -o SAMPLE6  
  
(kali㉿kali)-[~]  
$ ./SAMPLE6  
zsh: floating point exception ./SAMPLE6
```

Error Type: Division by zero

Error message: SIGFPE/ Arithmetic exception

Explanation: Dividing by zero causes a runtime crash.

Understanding Stack Frames

- Frame #0: The function where the crash actually occurred (divide).
- Frame #1: The function that called frame #0 (calculate).
- Frame #2: The top-level function that started the call sequence (main).

CASE 7 DEBUGGING ARRAY OUT OF BOUNDS ERRORS USING GDB

EXAMPLE CODE

```
#include <stdio.h>  
  
int main()  
{  
    int a[5];  
    int i;  
    for(i = 0; i <= 5; i++)  
    {  
        scanf("%d", &a[i]);  
    }
```

```
}  
return 0; }
```

```
(kali@kali)-[~]  
$ gcc -Wall -Wextra SAMPLE7.C -o SAMPLE7  
(kali@kali)-[~]  
$ ./SAMPLE7
```

Error Type: Array index error

Error Message: Array out-of-bounds Error

Explanation: Valid indices are 0 to 4. It will not give us output.

CASE 8 SEGMENTATION FAULTS (SIGSEGV) USING GDB

EXAMPLE CODE

```
#include <stdio.h>  
  
int main()  
{  
    int *p = NULL;  
    *p = 10;  
    printf("%d\n", *p);  
    return 0;  
}
```

DEBUGGING IN GDB

```
(kali@kali)-[~]  
$ gcc SAMPLE8.C -o SAMPLE8  
(kali@kali)-[~]  
$ ./SAMPLE8  
zsh: segmentation fault ./SAMPLE8
```

Error Type: NULL pointer dereference

Error Message: Segmentation fault

Explanation : Writing through a NULL pointer causes segmentation fault.

CORRECTED CODE

```
#include <stdio.h>

int main()
{
    int x;

    int *p = &x;
    *p = 10;
    printf("%d\n", *p);
    return 0;
}
```

CORRECTED OUTPUT

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The first command is \$ gcc SAMPLE8.C -o SAMPLE8. The second command is \$./SAMPLE8. The output is 10.

```
(kali@kali)-[~]
$ gcc SAMPLE8.C -o SAMPLE8
(kali@kali)-[~]
$ ./SAMPLE8
10
```

SYSTEMATIC DEBUUGING STRATEGY

To debug effectively, follow this 8-step methodology:

1. Observe Expected vs. Actual Output
2. Reproduce the Issue Consistently
3. Compile with Flags (-Wall -Wextra -g)
4. Isolate Suspicious Code Sections
5. Inspect State (printf or GDB breakpoints)

6. Apply a Single Minimal Fix
7. Recompile and Re-test
8. Verify with Edge Cases & Boundary Inputs